



aramex

Aramex Drones

Course: Operations Research

Instructor : Dr. Hamza Adeinat

**Students name: Omar Abdaljaleel , Abdel Rahman , Mohammed Darwish ,Mohammed
Kokash**

Date : 24/1/2025



Table of Contents

Introduction :	3
Problem Description :	3
Formulate the problem as a mathematical model	4
Indices :	4
Parameters :	4
Decision Variables:	5
Objective Function	5
Constraints	5
Assumptions and relevant parameters	6
Implement and solve model.....	6
Analyzation and interpret the results	15



Introduction :

The rapid growth of ecommerce and financial services has increased the demand for fast, reliable, and cost-efficient logistics solutions. Logistic providers such as **Aramex** are increasingly exploring innovative delivery methods, including **drone based transportation**, to improve operational efficiency and reduce delivery times within urban areas. In particular, the transportation of bank documents and small parcels between bank branches represents a suitable use case for drone technology due to its time sensitivity and relatively low payload requirements.

This project focuses on optimizing drone-based transportation operations for Aramex by applying **Operations Research (OR)** techniques. The objective is to support decision-making related to **transportation cost and capacity planning** rather than detailed route sequencing. The study considers multiple Aramex depots and a set of bank branches that generate pickup–delivery requests with known demand volumes.

A simplified optimization model is developed to determine the **minimum number of drone trips required for each pickup–delivery request**, while respecting drone capacity constraints and minimizing total transportation cost. To maintain alignment with the learning objectives of the Operations Research course, the model deliberately avoids complex vehicle routing formulations. Instead, routing paths are presented **for interpretation purposes only**, allowing the results to be easily understood and practically meaningful without introducing unnecessary computational complexity.

Realistic input data, including inter-location distances, order quantities, and operational parameters, are incorporated through structured CSV files. The results provide managerial insight into how drone capacity limitations and demand levels influence operational cost while highlighting the potential benefits of drone deployment in urban logistics networks.

Problem Description :

Aramex operates several logistics offices within the city that serve as drone deployment points. These depots are responsible for handling the transportation of banking documents and small parcels between different bank branches. Each transportation task consists of a **pickup operation at one bank branch and a delivery operation at another branch**, and each task is associated with a known number of orders.



Drones used by Aramex have a **limited carrying capacity per trip**, which restricts the number of orders that can be transported in a single flight. When the demand of a pickup–delivery request exceeds the drone’s capacity, multiple trips are required to fully satisfy that request. The transportation cost is assumed to be proportional to the travel distance and a fixed cost per kilometer.

The operational challenge addressed in this project is to determine **how many drone trips are required for each pickup–delivery request** in order to satisfy all demands at minimum total cost. The problem considers the distances between bank branches and Aramex depots, the number of orders associated with each request, and the drone capacity constraints.

To maintain clarity and computational simplicity the problem formula focus on **trip based cost optimization** rather than detailed route sequencing. Each trip is assumed to follow a logical operational path starting from an Aramex depot, traveling to the pickup branch, continuing to the delivery branch, and returning to the same depot. This routing is used solely for interpretation and cost estimation and is not optimized within the model.

Formulate the problem as a mathematical model

Indices :

N : Set of nodes (Aramex depot + Bank branches)

D : Depot node , index by 0

B : Bank branches

R : set of pick-up delivery requests

K : set of available drones

Parameters :

D_{ij} :Distance between node i and j (km)

C : cost per 1 kilometer (JD /km)

Q_r :Number of orders in request r

P_r :Pick-up node of request r



L_r : Delivery node of request r

Q : Drone capacity

$R_{\max}$: Maximum drone flying range per route (km)

Decision Variables:

Routing variable

$$x_{ijk} = \begin{cases} 1 & \text{if drone } k \text{ travels from node } i \text{ to node } j \\ 0 & \text{otherwise} \end{cases}$$

$$y_{rk} = \begin{cases} 1 & \text{if request } r \text{ is served by drone } k \\ 0 & \text{otherwise} \end{cases}$$

$$x_{ijk} \in \{0, 1\}$$

$$y_{rk} \in \{0, 1\}$$

Objective Function

Minimize total transportation cost

$$\min Z = \sum_{k \in K} \sum_{i \in N} \sum_{j \in N} c \cdot d_{ij} \cdot x_{ijk}$$

Constraints

Each drone starts and ends at the Aramex depot:

$$\sum_{j \in N} x_{0jk} = 1 \quad \forall k \in K$$

$$\sum_{i \in N} x_{i0k} = 1 \quad \forall k \in K$$

For every bank branch, inflow equals outflow:



$$\sum_{i \in N} x_{ijk} = \sum_{j \in N} x_{jik} \quad \forall j \in B, \forall k \in K$$

Each request must be served by **exactly one drone**:

$$\sum_{k \in K} y_{rk} = 1 \quad \forall r \in R$$

If a drone serves a request, it must visit both pickup and delivery nodes:

$$\sum_{j \in N} x_{p_rjk} \geq y_{rk} \quad \forall r \in R, \forall k \in K$$

$$\sum_{i \in N} x_{il_rk} \geq y_{rk} \quad \forall r \in R, \forall k \in K$$

Total orders carried by a drone must not exceed capacity:

$$\sum_{r \in R} q_r \cdot y_{rk} \leq Q \quad \forall k \in K$$

Total distance traveled by a drone must not exceed maximum range:

$$\sum_{i \in N} \sum_{j \in N} d_{ij} \cdot x_{ijk} \leq R_{\max} \quad \forall k \in K$$

Assumptions and relevant parameters

We make some assumptions , such as the orders between branches and the orders number.

Implement and solve model

We solve the problem using Microsoft Excel as below :



request_id	pickup_branch	delivery_branch	number_of_orders
1	Abdoun Branch	Shmeisani Branch	9
2	AL-Rawda Branch	Shmeisani Branch	6
3	Shafa Badran Branch	Jabal AL-Hussein Branch	2
4	Abdoun Branch	Tla' Al-Ali Branch	10
5	Shafa Badran Branch	Jabal AL-Hussein Branch	7
6	Jabal AL-Hussein Branch	Tla' Al-Ali Branch	9
7	AL-Rawda Branch	Abdoun Branch	7
8	Jabal AL-Hussein Branch	Shafa Badran Branch	9
9	Jabal AL-Hussein Branch	Shmeisani Branch	6
10	AL-Rawda Branch	Abdoun Branch	6

We assumed that this is the orders between the branches , from which branch to take the order and to which branch to deliver, and the number of orders .

Capacity per trip	Max range (km)	Cost per km
2	30	0.5

This table for the constrains , the capacity per trip is 2 orders , the max range for one charge is 30 km , and the cost per 1 km is 0.5 JD.

node_id	node_name	node_type
D0	Shmeisani Aramex Office	Depot
D1	Jubeiha Aramex Office	Depot
B1	Al-Taj Branch	Bank
B2	Shmeisani Bank Branch	Bank
B3	AL-Rawda Branch	Bank
B4	Tla' Al-Ali Branch	Bank
B5	Jabal AL-Hussein Branch	Bank

In this table, we choose the bank branches and Aramex office.



	Abdoun Branch	Shmesani Branch	AL-Rawda Branch	Tia' Al-Ali Branch	Jabal AL-Hussein Branch	Shafa Badran Branch	Shmeisani Aramex Office	Jubeiha Aramex Office
Abdoun Branch	--	2.60	5.50	4.50	3.50	11.20	3	9.5
Shmesani Branch	2.60	--	4.10	3.80	1.60	8.80	0.2	8.4
AL-Rawda Branch	5.50	4.10	--	1.80	5.30	6.40	4.9	3.5
Tia' Al-Ali Branch	4.50	3.80	1.80	--	5.20	8.10	4	4.6
Jabal AL-Hussein Branch	3.50	1.60	5.30	5.20	--	9.10	1.6	10.1
Shafa Badran Branch	11.20	8.80	6.40	8.10	9.10	--	8.7	6.1
Shmeisani Aramex Office	3	0.2	4.9	4	1.6	8.7	--	8.3
Jubeiha Aramex Office	9.5	8.4	3.5	4.6	10.1	6.1	8.3	0

In this table we select the distance between the branches of the bank.

Trip_Distance_km	Cost_per_trip	D.V
3	7.5	0
4.9	12.25	0
8.7	8.7	0
3	7.5	0
8.7	30.45	1
1.6	2.4	0
4.9	9.8	0
1.6	4.8	0
1.6	6.4	0
4.9	14.7	0

In this table we calculate the Trip distance using excel formulation:

=INDEX(\$N\$2:\$U\$9,MATCH("ShmeisaniAramexOffice",\$M\$2:\$M\$9,0),MATCH(B2,\$N\$1:\$U\$1,0))

This formulation match between the offices the branches in the distance table and with ShmeisaniAramexOffice and between the orders table.

The cost per trip = number of orders * trip distance

D.V = from the solver :

Solver Parameters

Set Objective:

To: ☐ Max ☒ Min ☐ Value Of:

By Changing Variable Cells:

Subject to the Constraints:

☒ Make Unconstrained Variables Non-Negative

Select a Solving Method:

Solving Method

Select the GRG Nonlinear engine for Solver Problems that are smooth nonlinear. Select the LP Simplex engine for linear Solver Problems, and select the Evolutionary engine for Solver problems that are non-smooth.



The objective is the Total cost , and it's a minimization problem , and the variable is binary.

Total cost (objective)
104.5

The output we got is 104.5 , so the problem is feasible with optimal solution that is 104.5.

Python :

```
# Distance Matrix
dist_data = {
    'Location': ['Abdoun Branch', 'Shmesani Branch', 'AL-Rawda Branch', 'Tla' Al-Ali Branch', 'Jabal AL-Hussein Branch', 'Shafa Badran Branch', 'Shmeisani Aramex Office', 'Jubeiha Aramex Office']
    'Abdoun Branch': [0, 2.6, 5.5, 4.5, 3.5, 11.2, 3, 9.5],
    'Shmesani Branch': [2.6, 0, 4.1, 3.8, 1.6, 8.8, 0.2, 8.4],
    'AL-Rawda Branch': [5.5, 4.1, 0, 1.8, 5.3, 6.4, 4.9, 3.5],
    'Tla' Al-Ali Branch': [4.5, 3.8, 1.8, 0, 5.2, 8.1, 4, 4.6],
    'Jabal AL-Hussein Branch': [3.5, 1.6, 5.3, 5.2, 0, 9.1, 1.6, 10.1],
    'Shafa Badran Branch': [11.2, 8.8, 6.4, 8.1, 9.1, 0, 8.7, 6.1],
    'Shmeisani Aramex Office': [3, 0.2, 4.9, 4, 1.6, 8.7, 0, 8.3],
    'Jubeiha Aramex Office': [9.5, 8.4, 3.5, 4.6, 10.1, 6.1, 8.3, 0]
}
df_dist = pd.DataFrame(dist_data).set_index('Location')

# Order Data
orders_data = {
    'request_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'pickup_branch': ['Abdoun Branch', 'AL-Rawda Branch', 'Shafa Badran Branch', 'Abdoun Branch', 'Shafa Badran Branch', 'Jabal AL-Hussein Branch', 'AL-Rawda Branch', 'Jabal AL-Hussein Branch', 'Shmeisani Branch', 'Shmeisani Branch'],
    'delivery_branch': ['Shmeisani Branch', 'Shmeisani Branch', 'Jabal AL-Hussein Branch', 'Tla' Al-Ali Branch', 'Jabal AL-Hussein Branch', 'Tla' Al-Ali Branch', 'Abdoun Branch', 'Shafa Badran Branch', 'Jabal AL-Hussein Branch', 'Shafa Badran Branch'],
    'number_of_orders': [9, 6, 2, 10, 7, 9, 7, 9, 6, 6]
}
df_orders = pd.DataFrame(orders_data)
```

Distance Matrix

A distance matrix is constructed to represent the pairwise travel distances between all relevant locations in the system. These locations include:

- Bank branches which act as pickup and delivery points
- Aramex offices which act as drone depots

The distance matrix stores the distance in kilometers between every pair of locations. Each row and column corresponds to a specific location, and the diagonal elements are zero representing zero distance from a location to itself. The matrix is symmetric, meaning the distance from location A to B is equal to the distance from B to A.

This matrix is converted to a Pandas DataFrame and indexed by location names to allow efficient lookup of distances between any two locations during the optimization process. The distance data serves as the basis for calculating transportation costs in the objective function.



Order (Demand) Data

The order data defines the set of transportation requests that must be fulfilled. Each request is characterized by:

- A unique request identifier
- A pickup branch, where orders originate
- A delivery branch, where orders must be delivered
- The number of orders associated with the request

This information is stored in a structured table where each row represents a single pickup–delivery request. The demand value indicates how many items must be transported and directly influences the number of drone trips required given the drone capacity constraints.

The order data is also stored in a Pandas DataFrame, enabling systematic iteration over requests when building the optimization model.

```
def get_distance(origin, dest):|
    try:
        return df_dist.loc[origin, dest]
    except KeyError:
        return 9999
```

To enable efficient and robust access to distance values during model construction, a distance lookup function `get_distance` is defined. This function retrieves the distance between any two locations using the previously constructed distance matrix.

The function takes two inputs: an origin location and a destination location. It attempts to return the corresponding distance value from the distance matrix. If the requested origin–destination pair is not found in the matrix (for example, due to naming inconsistencies or missing data), the function returns a very large penalty value.

This penalty mechanism ensures that infeasible or undefined transportation paths are automatically discouraged within the optimization model. By assigning a high cost to such paths, the solver avoids selecting them unless no feasible alternative exists. This approach improves model robustness and prevents runtime errors caused by missing or mismatched location names.



```
model = LpProblem("Aramex_Drone_Optimization", LpMinimize)
trip_vars = LpVariable.dicts("Trips", df_orders.index, lowBound=0, cat='Integer')
```

Model Initialization and Decision Variables

The optimization model is initialized as a **cost minimization problem**, representing the objective of minimizing total drone transportation cost. This is achieved by defining a linear programming model using a minimization objective.

A set of integer decision variables is then introduced to represent the operational decisions of the system. Specifically, for each pickup–delivery request, a decision variable is defined to represent the **number of drone trips required to satisfy that request**.

These variables are constrained to be non-negative integers, reflecting the practical reality that drone trips cannot be fractional. By defining one decision variable per request, the model directly captures the relationship between demand volume and required transportation effort.

From an Operations Research perspective, this formulation allows the problem to be expressed as an **integer linear programming (ILP)** model, where the decision variables represent discrete operational actions and the objective function evaluates their associated costs.

Mathematical Interpretation

Y_i = number of drone trips required for request i

Then:

$$Y_i \in \mathbb{Z}_{\geq 0}$$

These variables form the core decision set of the optimization model and are later used in both the objective function and the capacity constraints.



```
costs = []
for i in df_orders.index:
    pickup = df_orders.loc[i, 'pickup_branch']
    delivery = df_orders.loc[i, 'delivery_branch']
    dist = get_distance(pickup, delivery)
    costs.append(trip_vars[i] * dist * COST_PER_KM)

model += lpSum(costs)
```

Objective Function Definition

This part of the model defines the objective function, which represents the total transportation cost associated with all pickup delivery requests. The objective is to **minimize the overall operational cost** of drone deliveries.

For each request, the transportation cost is calculated as the product of three components:

- The number of drone trips required for the request
- The travel distance between the pickup branch and the delivery branch
- A fixed cost per kilometer representing operational cost

The model iterates over all transportation requests, retrieves the corresponding pickup and delivery locations, and determines the distance between them using the distance lookup function. This distance is then multiplied by the decision variable representing the number of trips and the cost per kilometer.

All individual request costs are aggregated using a summation operator to form the total cost function, which is then assigned as the objective of the optimization model.



```
for i in df_orders.index:
    demand = df_orders.loc[i, 'number_of_orders']
    model += trip_vars[i] * CAPACITY_PER_TRIP >= demand

status = model.solve()
```

This code segment defines the capacity constraint for each pickup delivery request, ensuring that the total carry capacity provided by the assigned number of drone trips is sufficient to satisfy the required demand. The constraint enforces that the number of trips multiplied by the drone capacity per trip is greater than or equal to the number of orders. After all constraints are defined, the optimization model is solved to obtain the optimal number of trips for each request.

```
# List of Depots
depots = ['Shmeisani Aramex Office', 'Jubeiha Aramex Office']

for i in df_orders.index:
    pickup = df_orders.loc[i, 'pickup_branch']
    delivery = df_orders.loc[i, 'delivery_branch']
    trips = int(trip_vars[i].value())

    d0_dist = get_distance(depots[0], pickup)
    d1_dist = get_distance(depots[1], pickup)

    if d0_dist <= d1_dist:
        start_depot = depots[0]
    else:
        start_depot = depots[1]
```

This code segment assigns a starting depot for each pickup–delivery request in order to present a realistic routing interpretation of the optimization results. A predefined list of Aramex depots is considered as potential starting and ending points for drone operations.



For each request, the distances between the pickup branch and each available depot are calculated using the distance lookup function.

The depot with the shortest distance to the pickup branch is selected as the starting depot for that request. This selection process is used solely for visualization and interpretation purposes and does not influence the optimization model or its objective value. The number of required trips is obtained from the optimized decision variables and is later combined with the selected depot to display an operationally intuitive route.

This approach enables the results to be presented in a practical and understandable manner while maintaining the simplicity of the optimization formulation and avoiding explicit routing decision variables.

```
[Request ID: 5]
Route: Jubeiha Aramex Office -> Shafa Badran Branch -> Jabal AL-Hussein Branch -> Jubeiha Aramex Office
Trips Required: 4

[Request ID: 6]
Route: Shmeisani Aramex Office -> Jabal AL-Hussein Branch -> Tla' AL-Ali Branch -> Shmeisani Aramex Office
Trips Required: 5

[Request ID: 7]
Route: Jubeiha Aramex Office -> AL-Rawda Branch -> Abdoun Branch -> Jubeiha Aramex Office
Trips Required: 4

[Request ID: 8]
Route: Shmeisani Aramex Office -> Jabal AL-Hussein Branch -> Shafa Badran Branch -> Shmeisani Aramex Office
Trips Required: 5

[Request ID: 9]
Route: Shmeisani Aramex Office -> Jabal AL-Hussein Branch -> Shmeisani Branch -> Shmeisani Aramex Office
Trips Required: 3

[Request ID: 10]
Route: Jubeiha Aramex Office -> AL-Rawda Branch -> Abdoun Branch -> Jubeiha Aramex Office
Trips Required: 3

*****
Total Operational Cost: 104.05000000000001 JOD
*****
```



Analyzation and interpret the results

The optimization model successful determined the minimum number of drone trips required to satisfy all pickup delivery request while minimizg total transportation cost. All capacity constraints were satisfied, and an optimal solution was obtained.

Trip Allocation Analysis

The results indicate that the number of required trips for each request is directly driven by the ratio between demand volume and drone capacity. Requests with higher order quantities require a larger number of trips, while smaller demands can be served with fewer flights.

For example, Requests 1, 4, 6, and 8 each require five trips, reflecting high demand volumes relative to the drone's limited carrying capacity. In contrast, Requests 3 and 9 require only one and three trips respectively, due to lower order quantities. This demonstrates the effectiveness of the capacity constraint in enforcing realistic operational limitations.

Routing Interpretation

For interpretability, each request is associated with a logical operational route of the form:

Aramex Depot → Pickup Branch → Delivery Branch → Aramex Depot

The starting depot is selected based on proximity to the pickup branch, providing a realistic operational assumption without affecting the optimization results. For instance, requests originating near the Jubeiha area are assigned to the Jubeiha Aramex Office, while centrally located pickups are served from the Shmeisani Aramex Office.

It is important to note that these routes are presented for explanatory purposes only. The optimization model itself does not optimize routing sequences or depot assignments, but rather focuses on minimizing cost through efficient trip allocation.

Cost Implications

The total optimized operational cost for serving all requests is **104.05 JOD**. This value reflects the minimum cost associated with transporting all orders between pickup and delivery branches under the defined assumptions.



Requests with both high demand and longer distances contribute most significantly to the total co routes involving Shafa Badran and Jabal Al-Hussein branches incur higher costs due to greater travel distances, whereas shorter routes such as Jabal Al-Hussein to Shmeisani Branch generate lower costs despite multiple trips.

This highlights how transportation distance and demand volume jointly influence operational cost in drone-based logistics.

Managerial Insights

The results provide several practical insights:

- Drone capacity plays a critical role in determining the number of required trips and overall cost.
- High demand routes may benefit from capacity upgrades or demand consolidation.
- Locating depots closer to high-frequency pickup locations can reduce operational costs.
- Even without complex routing optimization, meaningful cost-minimization decisions can be achieved through trip-based modeling.